Abdelmalek Essaâdi University
Faculty of Sciences and Technologies Tangier

Master of Science and Technology
IT Security and Big Data

Lab 4 - Thin Provisioning & Monitoring du Stockage

Analyse pratique du provisioning Thin/Thick, supervision et conversion des disques virtuels sous ESXi

Student : Azzaoui Mohamed Omar
Instructor : Mohamed BEN AHMED

Academic Year : 2025–2026

Introduction

Ce laboratoire porte sur la gestion du stockage dans un environnement ESXi, avec un accent particulier sur le *thin provisioning*, le *thick provisioning* et la surveillance des entrées/sorties disque. L'objectif n'était pas seulement d'exécuter des commandes, mais surtout de vérifier expérimentalement la différence entre l'espace **déclaré** à une machine virtuelle et l'espace **réellement consommé** sur le datastore.

Le travail réalisé a couvert quatre idées principales : la création de disques virtuels de formats différents, l'observation du comportement dynamique d'un disque thin lors de l'écriture, la surveillance du stockage avec `esxtop` et `esxcli`, puis la conversion d'un disque thin vers un disque *thick eager zeroed*. La partie `PowerCLI` et la conversion *Thick* → *Thin* par *Storage vMotion* seront présentées de manière théorique, car notre environnement de travail reposait sur un hôte ESXi standalone sans vCenter.

Objectifs du laboratoire

À partir des consignes fournies, les objectifs visés dans ce TP étaient les suivants :

- créer des disques virtuels en mode thin et thick ;
- comparer l'espace alloué logiquement et l'espace réellement utilisé ;
- observer la croissance d'un disque thin après des écritures réelles ;
- surveiller l'activité de stockage en temps réel ;
- convertir un disque thin vers un format *thick eager zeroed* ;
- identifier les limites et les risques liés au sur-engagement du stockage.

Rappel théorique

Thin Provisioning

Dans le mode *thin provisioned*, le disque virtuel annonce une capacité donnée à la machine virtuelle, mais l'espace physique n'est consommé sur le datastore qu'au fur et à mesure des écritures. Ce mode est pratique pour les laboratoires, les environnements de test et les machines peu actives. Son inconvénient principal est le risque d'*over-commitment* : plusieurs disques peuvent annoncer une grande capacité totale sans que le datastore dispose réellement de cet espace.

Thick Provisioning

Dans le mode *thick*, l'espace est alloué immédiatement. Dans le cas *eager zeroed thick*, les blocs sont en plus initialisés à zéro dès la création. Ce format consomme plus d'espace dès le départ, mais il apporte un comportement plus prévisible et de meilleures garanties pour les charges sensibles.

Idée clé du TP

Le point central du laboratoire est donc le suivant : **la taille logique d'un disque virtuel n'est pas toujours égale à la place qu'il occupe réellement sur le datastore**. Toute la démarche pratique du TP consiste à rendre cette différence visible, mesurable et interprétable.

Environnement de travail

Le laboratoire a été réalisé sur un hôte ESXi avec un datastore principal nommé `iSCSI-VMFS-DS01`. Une machine virtuelle Ubuntu existante a servi de système invité pour le montage des disques et pour la génération de charges d'écriture. L'ensemble du travail a été mené directement depuis l'ESXi Host Client et une session SSH sur l'hôte, puis vérifié depuis Ubuntu.

Étape 1 – Création d'un disque Thin Provisioned

Pour éviter une saturation prématurée du datastore, une taille de `20G` a été retenue à la place des `40G` proposés dans l'énoncé. La création du disque thin a été réalisée avec `vmkfstools` après préparation du répertoire de destination.

```
mkdir -p /vmfs/volumes/iSCSI-VMFS-DS01/VM-Thin
vmkfstools -c 20G -d thin \
/vmfs/volumes/iSCSI-VMFS-DS01/VM-Thin/VM-Thin.vmdk
```

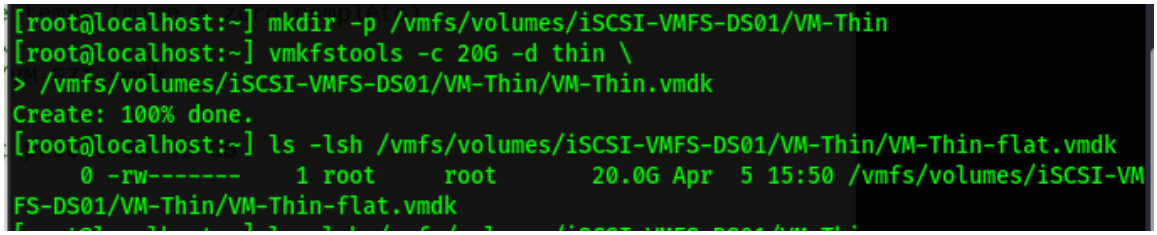


Figure 1 – thin-provisioning.png

Dans cette capture, la création du disque se termine correctement avec `Create: 100% done`. La commande `ls -lsh` affiche ensuite le fichier `VM-Thin-flat.vmdk` avec une taille logique de `20.0G`. À ce stade, cette observation pourrait laisser croire que l'espace est déjà entièrement occupé, mais ce n'est pas le bon indicateur pour juger le comportement du thin provisioning sous VMFS.

La bonne approche consiste à utiliser `du -sh`, qui mesure l'espace réellement consommé sur le datastore.

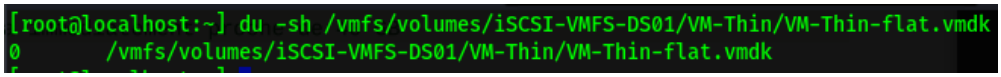
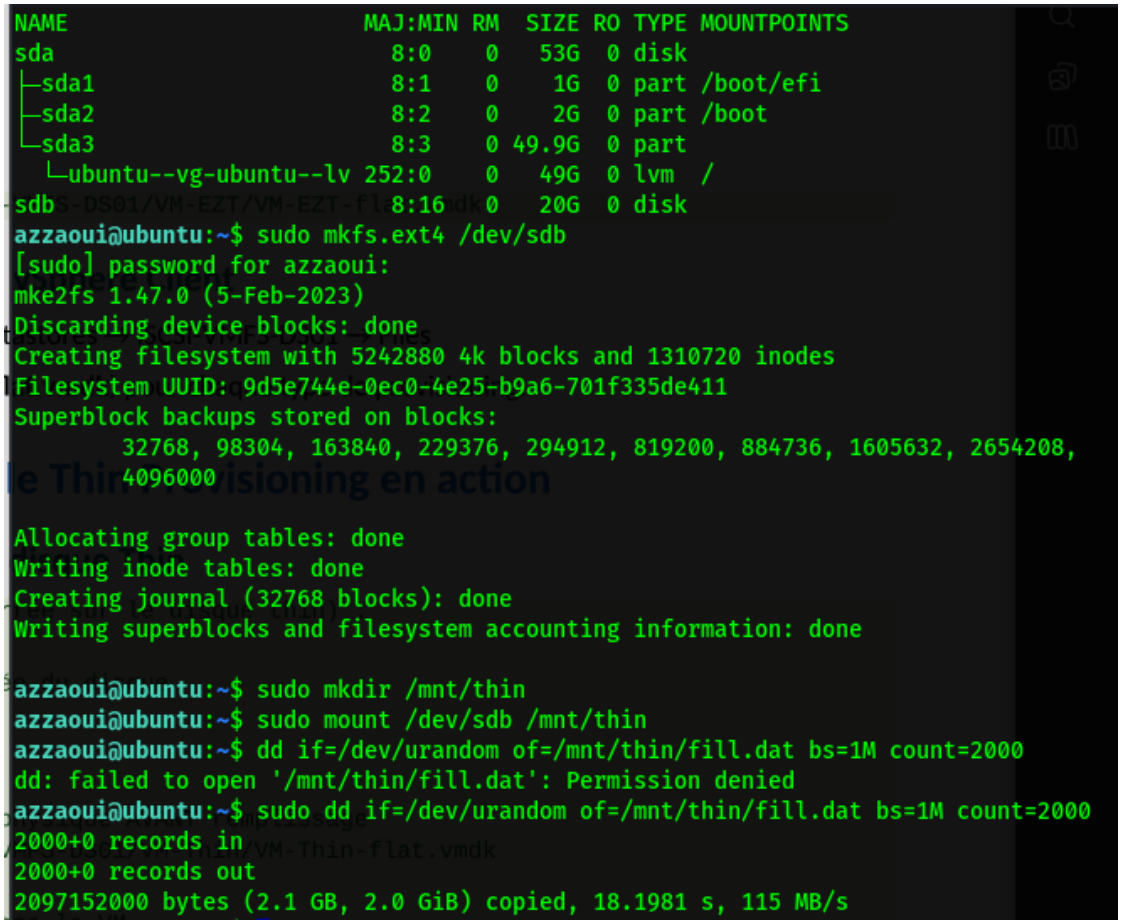


Figure 2 – du-thin-before.png

Cette capture est essentielle : le résultat renvoyé est `0`, ce qui confirme que le disque thin existe logiquement, mais qu'il n'occupe pratiquement aucun espace physique avant l'écriture de données réelles. C'est le premier résultat expérimental du TP.

Étape 2 – Mise en service du disque thin dans Ubuntu

Le disque créé côté ESXi ne peut pas être observé depuis l'OS invité tant qu'il n'a pas été attaché à la VM. Après ajout du disque existant à la machine Ubuntu via l'ESXi Host Client, la vérification dans Ubuntu a montré la présence d'un nouveau périphérique `/dev/sdb` de `20G`.



```
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                         8:0    0   53G  0 disk
├─sda1                      8:1    0    1G  0 part /boot/efi
├─sda2                      8:2    0    2G  0 part /boot
└─sda3                      8:3    0 49.9G  0 part
  └─ubuntu--vg-ubuntu--lv 252:0    0   49G  0 lvm  /
sdb                         8:16   0   20G  0 disk
azzaoui@ubuntu:~$ sudo mkfs.ext4 /dev/sdb
[sudo] password for azzaoui:
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 5242880 4k blocks and 1310720 inodes
Filesystem UUID: 9d5e744e-0ec0-4e25-b9a6-701f335de411
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
        4096000

Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done

azzaoui@ubuntu:~$ sudo mkdir /mnt/thin
azzaoui@ubuntu:~$ sudo mount /dev/sdb /mnt/thin
azzaoui@ubuntu:~$ dd if=/dev/urandom of=/mnt/thin/fill.dat bs=1M count=2000
dd: failed to open '/mnt/thin/fill.dat': Permission denied
azzaoui@ubuntu:~$ sudo dd if=/dev/urandom of=/mnt/thin/fill.dat bs=1M count=2000
2000+0 records in
2000+0 records out
2097152000 bytes (2.1 GB, 2.0 GiB) copied, 18.1981 s, 115 MB/s
```

Figure 3 – thin-mount-ubuntu.png

Cette capture regroupe plusieurs opérations importantes :

- `lsblk` confirme l'apparition du disque `sdb` de `20G` ;
- `mkfs.ext4 /dev/sdb` initialise un système de fichiers sur ce disque ;
- le point de montage `/mnt/thin` est créé puis utilisé ;
- une première tentative d'écriture avec `dd` sans privilèges échoue avec `Permission denied` ;
- l'utilisation de `sudo` corrige immédiatement le problème ;
- l'écriture de `2000 MiB` de données aléatoires s'effectue correctement.

Le choix de `/dev/urandom` est justifié : il produit des données non triviales, difficiles à compresser ou à optimiser par le stockage. Ainsi, la croissance du disque thin reflète bien une occupation réelle du datastore.

```
azzaoui@ubuntu:~$ df -h /mnt/thin
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb         20G  2.0G   17G  11% /mnt/thin
azzaoui@ubuntu:~$
```

Figure 4 – mntTHINused2GB-ubuntu.png

La commande `df -h /mnt/thin` confirme que le système invité voit un disque de `20G`, dont environ `2.0G` sont maintenant utilisés. Côté Ubuntu, l'écriture est donc bien effective ; il reste à vérifier que cette activité se traduit aussi par une augmentation de l'espace consommé sur ESXi.

```
[root@localhost:~] du -sh /vmfs/volumes/iSCSI-VMFS-DS01/VM-Thin/VM-Thin-flat.vmdk
2.1G    /vmfs/volumes/iSCSI-VMFS-DS01/VM-Thin/VM-Thin-flat.vmdk
[root@localhost:~]
```

Figure 5 – thin-after.png

Le résultat de `du -sh` après l'écriture affiche `2.1G`. Cette valeur est cohérente avec les `2000 MiB` écrits depuis Ubuntu. Cette capture constitue la démonstration principale du *thin provisioning en action* : le disque n'occupe pas sa taille maximale dès sa création, mais il grandit en fonction des données effectivement stockées.

Analyse de l'observation

Les mesures obtenues permettent de conclure que :

- la taille logique du disque reste `20G` pour le système invité ;
- l'espace réel sur le datastore passe de `0` à environ `2.1G` après l'écriture ;
- le mode thin est donc bien dynamique ;
- ce comportement est avantageux pour économiser l'espace, mais il impose une surveillance active du datastore.

Étape 3 – Création d'un disque Thick Eager Zeroed

La seconde partie pratique a consisté à créer un disque *eager zeroed thick* de même taille afin de comparer directement les deux approches.

```
mkdir -p /vmfs/volumes/iSCSI-VMFS-DS01/VM-Thick
vmkfstools -c 20G -d eagerzeroedthick \
/vmfs/volumes/iSCSI-VMFS-DS01/VM-Thick/VM-Thick.vmdk
```

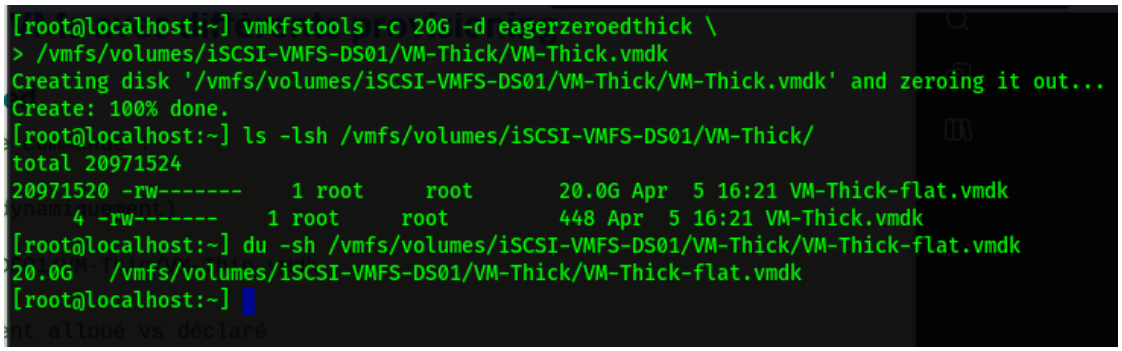


Figure 6 – thinkDiskProvisioninfESXI.png

Cette capture montre une différence immédiate avec le disque thin. Le message `Creating disk ... and zeroing it out...` indique que l'allocation et la mise à zéro sont réalisées dès la création. Les vérifications `ls -lsh` et `du -sh` renvoient toutes deux `20.0G`. Autrement dit, le disque thick eager zeroed consomme tout son espace dès le départ, indépendamment du volume de données réellement écrites.

Comparaison Thin vs Thick

Critère	Thin Provisioned	Thick Eager Zeroed
Allocation initiale	Progressive, selon les écritures	Immédiate et complète
Occupation observée au départ	0 sur le datastore	20G sur le datastore
Croissance	Oui, en fonction des données réelles	Non, l'espace est déjà réservé
Avantage principal	Économie d'espace	Prévisibilité et disponibilité immédiate
Risque principal	Saturation du datastore en cas d'over-commitment	Consommation d'espace plus importante

Cette comparaison rend visible le compromis classique entre souplesse et sécurité. Le thin optimise l'espace disponible, tandis que le thick privilégie la stabilité et l'anticipation des besoins.

Étape 4 – Monitoring avec `esxtop` et `esxcli`

La partie monitoring avait pour but d'observer l'activité du stockage pendant l'exécution des écritures. Deux vues ont été utilisées dans `esxtop` : la vue `u` pour les adaptateurs de stockage et la vue `v` pour les disques virtuels.

Figure 7 – esxtop-storageAdapter.png

Dans la vue *Storage Adapter*, on retrouve les colonnes classiques de supervision : `CMDS/s`, `READS/s`, `WRITES/s`, `MBREAD/s`, `MBWRTN/s`, `DAVG/cmd`, `KAVG/cmd` et `GAVG/cmd`. Au moment de cette capture, les valeurs sont nulles ou très faibles, ce qui correspond à une phase sans charge notable. L'intérêt de cette vue est de suivre la latence moyenne vue par le device. En pratique, un `DAVG/cmd` supérieur à `25 ms` constitue un signal d'alerte.

```
4:26:33pm up  1:29, 648 worlds, 1 VMs, 2 vCPUs; CPU load average: 0.04, 0.03, 0.04

        GID VMNAME           VDEVNAME NVDISK   CMDS/s  READS/s WRITES/s MBREAD/s MBWRTN/s LAT/rd LAT/wr
      11624 Ubuntu                  -      2     0.00     0.00     0.00     0.00     0.00  0.000  0.000
```

Figure 8 – esxtopVirtual.png

La vue *Virtual Disk* permet de relier l'activité I/O à la machine virtuelle concernée. On y voit la VM `Ubuntu`, le nombre de disques virtuels visibles, ainsi que les colonnes `CMDS/s`, `READS/s`, `WRITES/s`, `MBREAD/s` et `MBWRTN/s`. Cette vue est très utile pour vérifier quelle VM génère la charge observée.

```
Every 2s: esxcli storage core device stats get | head -40

mpx.vmhba0:C0:T1:L0
   Device: mpx.vmhba0:C0:T1:L0
   Successful Commands: 58424
   Blocks Read: 1077248
   Blocks Written: 57397431
   Read Operations: 15760
   Write Operations: 41675
   Reserve Operations: 401
   Reservation Conflicts: 0
   Failed Commands: 5
   Failed Blocks Read: 0
   Failed Blocks Written: 0
   Failed Read Operations: 0
   Failed Write Operations: 0
   Failed Reserve Operations: 0

mpx.vmhba0:C0:T0:L0
   Device: mpx.vmhba0:C0:T0:L0
   Successful Commands: 19079
   Blocks Read: 245486
   Blocks Written: 129523
   Read Operations: 9429
   Write Operations: 9299
   Reserve Operations: 0
   Reservation Conflicts: 0
   Failed Commands: 5
   Failed Blocks Read: 0
   Failed Blocks Written: 0
   Failed Read Operations: 0
   Failed Write Operations: 0
   Failed Reserve Operations: 0
```

Figure 9 – statsperdevice.png

La commande `watch -n 2 'esxcli storage core device stats get | head -40'` a été utilisée comme complément à `esxtop`. Cette capture montre les statistiques cumulées par device : nombre de commandes réussies, blocs lus, blocs écrits, opérations de lecture et

d'écriture, ainsi que les échecs éventuels. On observe ici un volume d'écritures nettement supérieur aux lectures sur le chemin `mpx.vmhba0:C0:T1:L0`, ce qui est cohérent avec les tests effectués à l'aide de `dd` dans Ubuntu.

Lecture des indicateurs

Les indicateurs suivis pendant le TP ont la signification suivante :

- `DAVG/cmd` : latence moyenne au niveau du périphérique de stockage ;
- `KAVG/cmd` : temps passé dans le noyau VMkernel ;
- `GAVG/cmd` : latence totale perçue par la VM ;
- `MBWRTN/s` : débit d'écriture ;
- `CMDS/s` : nombre de commandes d'E/S par seconde.

Même si les captures ont été prises dans une phase relativement calme, elles montrent bien la méthodologie de supervision attendue par le laboratoire : identifier la source des entrées/sorties, vérifier que les écritures passent correctement, et surveiller les latences pour détecter une saturation éventuelle.

Étape 5 – Conversion Thin → Thick Eager Zeroed

La conversion demandée dans le sujet a été réalisée avec `vmkfstools -i`. Comme le disque thin était utilisé par la VM Ubuntu, la machine a d'abord été arrêtée avec `vim-cmd`, puis le disque thin a été cloné vers un nouveau VMDK au format *eagerzeroedthick*. La logique de la commande était la suivante :

```
vmkfstools -i /vmfs/volumes/iSCSI-VMFS-DS01/VM-Thin/VM-Thin.vmdk \
/vmfs/volumes/iSCSI-VMFS-DS01/VM-Thick/VM-Thin-Converted.vmdk \
-d eagerzeroedthick
```

La vérification côté ESXi a montré que le fichier `VM-Thin-Converted-flat.vmdk` occupait désormais `20.0G`, ce qui confirme que le disque converti est devenu un disque thick entièrement alloué. La capture suivante documente précisément l'arrêt de la VM, le clonage vers `VM-Thin-Converted.vmdk` puis la suppression de l'ancien disque thin.

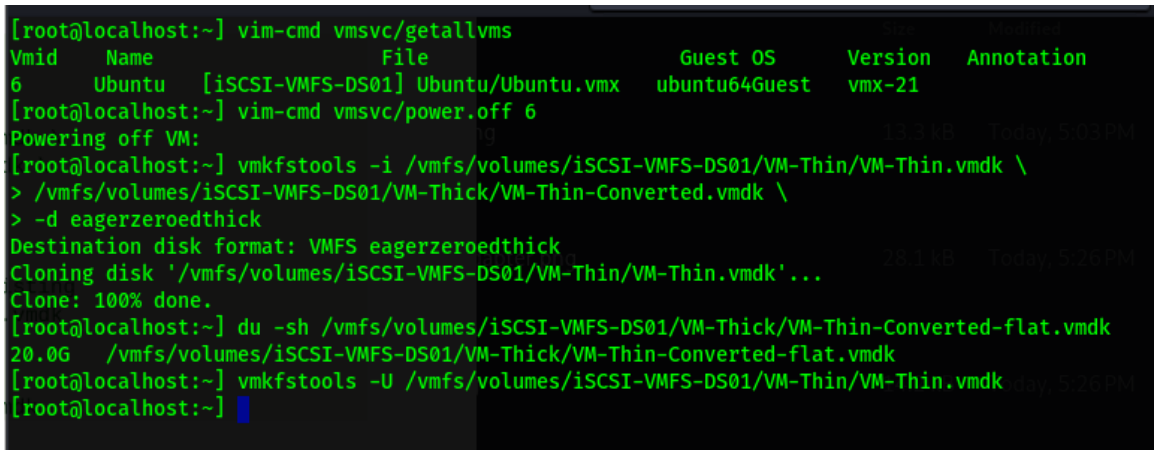
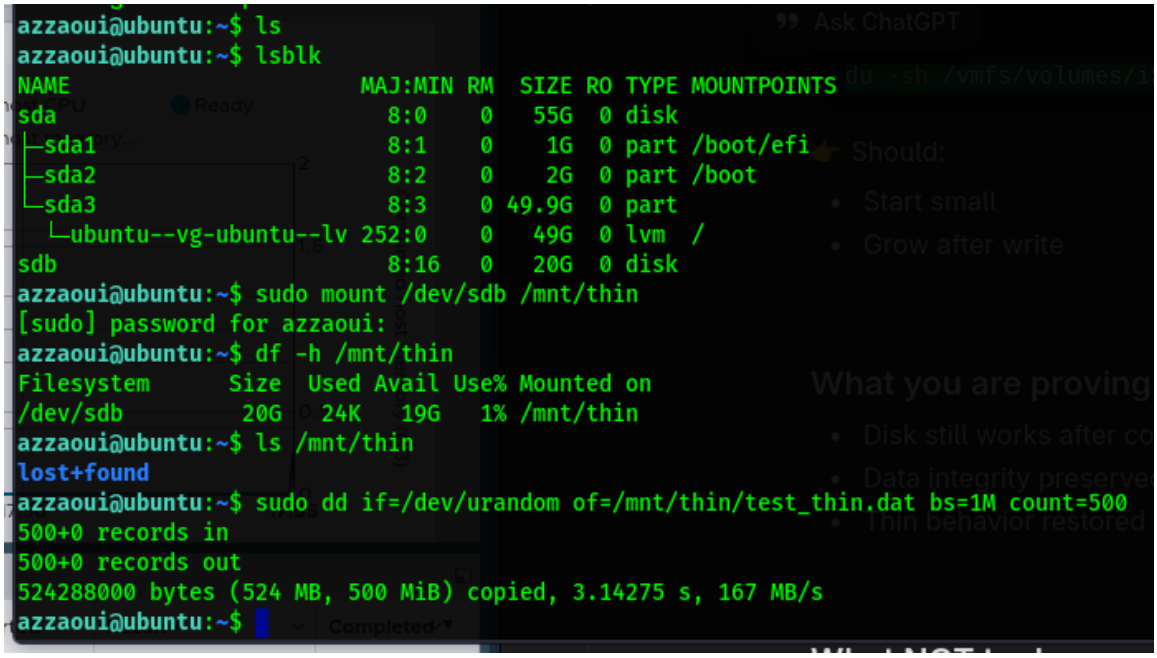


Figure 10 – thin-thick-Zeroed.png

Après rattachement à la VM Ubuntu, le disque converti a été remonté pour valider sa continuité fonctionnelle.



```
azzaoui@ubuntu:~$ ls
azzaoui@ubuntu:~$ lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                         8:0    0   55G  0 disk
├─sda1                      8:1    0    1G  0 part /boot/efi
├─sda2                      8:2    0    2G  0 part /boot
└─sda3                      8:3    0 49.9G  0 part
  └─ubuntu--vg-ubuntu--lv 252:0    0   49G  0 lvm  /
sdb                         8:16   0   20G  0 disk
azzaoui@ubuntu:~$ sudo mount /dev/sdb /mnt/thin
[sudo] password for azzaoui:
azzaoui@ubuntu:~$ df -h /mnt/thin
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb         20G   24K   19G   1% /mnt/thin
azzaoui@ubuntu:~$ ls /mnt/thin
lost+found
azzaoui@ubuntu:~$ sudo dd if=/dev/urandom of=/mnt/thin/test_thin.dat bs=1M count=500
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 3.14275 s, 167 MB/s
azzaoui@ubuntu:~$
```

Figure 11 – thin-thickzeroed ubuntu.png

Cette capture confirme plusieurs points :

- Ubuntu détecte toujours un disque secondaire `/dev/sdb` de `20G` ;
- le montage sur `/mnt/thin` s'effectue correctement ;
- l'espace utilisé est redevenu très faible car les gros fichiers de test avaient été supprimés avant la conversion ;
- une nouvelle écriture de `500 MiB` réussit sans erreur ;
- la conversion n'a donc pas rendu le disque inutilisable du point de vue du système invité.

Remarque importante sur la saturation du datastore

Au cours des manipulations, un incident de démarrage de la VM a été rencontré à cause d'un manque d'espace libre sur le datastore. ESXi n'a pas pu étendre le fichier de swap `.vswp` au démarrage. Cet incident illustre parfaitement le risque réel lié à l'accumulation de disques thick et à l'absence de surveillance active de l'espace restant. Même dans un environnement de laboratoire, la gestion du stockage ne se limite donc pas à la création des VMDK : il faut aussi anticiper l'espace requis pour les fichiers annexes du runtime ESXi.

Partie théorique – Monitoring avec PowerCLI

La partie `PowerCLI` du sujet n'a pas été exécutée directement dans notre environnement, car celui-ci reposait sur un hôte ESXi standalone, sans chaîne complète d'automatisation

Windows/vCenter. Néanmoins, le rôle de `PowerCLI` reste central dans une infrastructure VMware de production.

`PowerCLI` permet notamment :

- d'obtenir un rapport global sur les datastores ;
- de calculer automatiquement le pourcentage d'occupation ;
- de lister les disques par VM avec leur format de stockage ;
- d'automatiser des vérifications périodiques et des alertes de seuil.

L'intérêt principal de cette partie est de passer d'une observation ponctuelle à une approche d'administration préventive. Par exemple, un script PowerCLI peut déclencher une alerte lorsque l'occupation d'un datastore dépasse `80%`. Ce point devient essentiel lorsque l'on utilise des disques thin, car la croissance n'est pas toujours visible immédiatement depuis la VM elle-même.

Partie théorique – Conversion Thick → Thin

Le sujet propose une conversion *Thick* → *Thin* via *Storage vMotion*. Cette méthode suppose un environnement plus complet avec vCenter et plusieurs options de migration de stockage. Dans notre cas, cette opération n'a pas été réalisée de manière native, car l'hôte ESXi standalone ne permet pas de reproduire le scénario de *Storage vMotion* tel qu'il est présenté dans l'énoncé.

Sur le plan technique, l'idée reste la même : il s'agit de cloner ou migrer un disque en changeant son format de stockage. Dans une infrastructure disposant de vCenter, cela permettrait de convertir à chaud ou via une migration de stockage un disque thick vers un disque thin, tout en conservant le contenu de la VM.

Autrement dit, chaque capture n'a pas été utilisée comme simple illustration, mais comme une preuve expérimentale d'un comportement précis du stockage virtuel.

Conclusion

Ce laboratoire a permis de valider de manière concrète la différence fondamentale entre un disque *thin provisioned* et un disque *thick eager zeroed*. Le disque thin a montré un comportement dynamique : il n'occupait aucun espace réel au départ, puis sa taille effective a augmenté après l'écriture de données dans Ubuntu. À l'inverse, le disque thick eager zeroed a réservé immédiatement toute sa capacité sur le datastore.

La phase de monitoring a montré les outils à utiliser pour suivre les performances du stockage et interpréter l'activité I/O : `esxtop` pour la latence et les débits, `esxcli` pour les statistiques par device, et théoriquement `PowerCLI` pour la supervision automatisée. Enfin, la conversion Thin → Thick a confirmé qu'un simple changement de format a un impact direct sur la consommation réelle d'espace.

Au-delà des commandes exécutées, la leçon la plus importante de ce TP est la suivante : **le gain d'espace offert par le thin provisioning reste utile uniquement si l'administrateur surveille en permanence l'occupation réelle du datastore**. Sans cette vigilance, l'économie initiale peut rapidement se transformer en panne de stockage.